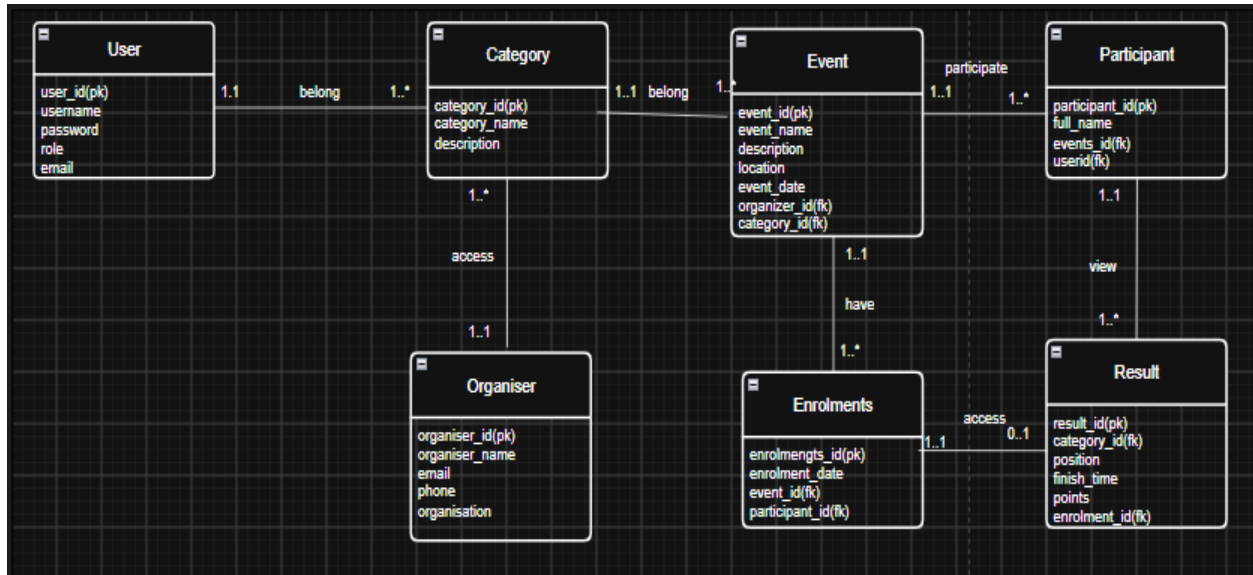


ST10497766

SECTION A -ERD



SECTION B - API Endpoint Plan

HTTP method	Route	Description	Role required	Request body	Expected response
POST	/api/auth/register	Registers a new user account as an organiser or participant	Public	{ "full_name": "...", "email": "...", "password": "...", "role": "participant" }	201 Created – user registered. 400 – invalid data/email already exists.
POST	/api/auth/login	Authenticates a registered user and provides access to	Public	{ "email": "...", "password": "..." }	200 OK – login successful with token. 401 – invalid

		the system.			credential s.
GET	/api/users/profile	Retrieves the profile information of the currently logged-in user.	Authenticated	None	200 OK – user profile. 401 – not authenticated.
PUT	/api/users/profile	Updates the logged-in user's profile information.	Authenticated	{ "full_name": "...", "email": "...", "cell_number": "..."}	200 OK – profile updated. 400 – invalid data
GET	/api/events	Retrieves a list of upcoming RaceDay events.	Public	Query parameters optional	200 OK – list of events.
GET	/api/events/{id}	Retrieves details of a specific event.	Public	None	200 OK – event details. 404 – event not found.

POST	/api/events	Creates a new event that participants can register for.	Organiser	{ "event_name": "...", "event_type": "...", "description": "...", "event_date": "...", "location": "..." }	201 Created – event created. 400 – invalid data.
PUT	/api/events/{id}	Updates the details of an event managed by an organiser.	Organiser	Event fields to update	200 OK – event updated. 404 – event not found.
DELETE	/api/events/{id}	Deletes an event created by the organiser.	Organiser	None	204 No Content – event deleted. 404 – event not found.
GET	/api/categories	Retrieves all event categories available in the system.	Public	None	200 OK – list of categories.
GET	/api/categories/{id}	Retrieves a specific category and its details.	Public	None	200 OK – category details. 404 – category not found.
POST	/api/categories	Creates a new event category.	Organiser	{ "category_name": "...", "description": "..." }	201 Created – category created. 400 –

				ion": "..."} }	invalid data.
PUT	/api/categories/{id}	Updates an existing event category.	Organiser	Category fields	200 OK – category updated. 404 – category not found.
DELETE	/api/categories/{id}	Removes an event category that is no longer required.	Organiser	None	204 No Content – category deleted. 404 – category not found.
POST	/api/events/{eventId}/enrolments	Registers a participant for a specific event. This represents the REGISTRATION relationship in the ERD.	Participant	{ "participant_id": 1, "payment_status": "paid", "amount_paid": 150.00 }	201 Created – registration successful. 409 – participant already registered. 404 – event/participant not found.
GET	/api/events/{eventId}/enrolments	Retrieves all participants registered for an event.	Organiser	None	200 OK – list of registered participants
DELETE	/api/events/{eventId}/enrolments/{participantId}	Cancels a participant's registration	Participant	None	204 No Content – registration cancelled.

		for an event.			404 – registration not found
GET	/api/participants/{id}	Retrieves a participant's information and event participation details.	Authenticated	None	200 OK – participant details. 404 – participant not found.
GET	/api/participants/{id}/results	Retrieves a participant's results across events	Authenticated	None	200 OK – participant result history
POST	/api/events/{eventId}/results	Records a participant's result for an event and category	Organiser	{ "participant_id": 1, "category_id": 2, "position": 5, "finish_time": "01:25:30", "points": 95 }	201 Created – result recorded. 400 – invalid result. 404 – participant/event not found.
GET	/api/events/{eventId}/results	Retrieves all results recorded for a specific event.	Public	None	200 OK – event results.
GET	/api/events/{eventId}/results/{participantId}	Retrieves a specific participant's result for an event.	Public	None	200 OK – result details. 404 –

					result not found
PUT	/api/results/{id}	Updates a participant's result, such as position, finish time or points.	Organiser	{ "position": 3, "finish_time": "01:18:20", "points": 100 }	200 OK – result updated. 404 – result not found.
DELETE	/api/results/{id}	Deletes an incorrectly recorded result	Organiser	None	204 No Content – result deleted. 404 – result not found.
GET	/api/participants/{id}/performance-history	Retrieves a participant's historical performance across completed	Authenticated	None	200 OK – performance history. 404 – participant not found.

Section c – Sql script

```
-- =====
-- RACEDAY EVENT MANAGEMENT SYSTEM
-- FULL DATABASE SCHEMA AND SAMPLE DATA
-- SQL SERVER / SSMS
-- =====
```

```
create database raceday;
```

```
use raceday;
```

```
-- 1. USER TABLE
```

```
create table [user] (
```

```
    user_id int identity(1,1) primary key,
```

```
    username varchar(50) not null unique,
```

```
    password_hash varchar(255) not null,
```

```
    role varchar(20) not null,
```

```
    email varchar(100) not null unique,
```

```
    created_at datetime not null default getdate(),
```

```
    constraint chk_user_role
```

```
        check (role in ('organiser', 'participant'))
```

```
);
```

```
-- 2. ORGANISER TABLE
```

```
create table organiser (
```

```
    organiser_id int identity(1,1) primary key,
```

```
    user_id int not null unique,
```

```
    organiser_name varchar(100) not null,
```

```
    email varchar(100) not null unique,
```

```
    phone varchar(20) not null,
```

```
    organisation varchar(150) not null,
```

```
constraint fk_organiser_user  
    foreign key (user_id)  
    references [user](user_id)  
);
```

-- 3. PARTICIPANT TABLE

```
create table participant (  
    participant_id int identity(1,1) primary key,  
    user_id int not null unique,  
    full_name varchar(100) not null,  
    email varchar(100) not null unique,  
    cell_number varchar(20) not null,  
    date_of_birth date not null,  
    gender varchar(20) not null,  
  
    constraint fk_participant_user  
        foreign key (user_id)  
        references [user](user_id),  
  
    constraint chk_participant_gender  
        check (gender in ('Male', 'Female', 'Other'))  
);
```


-- 4. EVENT TABLE

```
create table event (  
    event_id int identity(1,1) primary key,  
    organiser_id int not null,  
    event_name varchar(150) not null,  
    event_type varchar(30) not null,  
    description varchar(500) null,  
    event_date date not null,  
    location varchar(150) not null,  
    start_time time not null,  
    end_time time not null,  
    status varchar(20) not null default 'upcoming',  
  
    constraint fk_event_organiser  
        foreign key (organiser_id)  
        references organiser(organiser_id),  
  
    constraint chk_event_type  
        check (event_type in ('Running', 'Walking', 'Cycling')),  
  
    constraint chk_event_status  
        check (status in ('upcoming', 'active', 'completed', 'cancelled')),  
  
    constraint chk_event_time  
        check (end_time > start_time)
```

```
);
```

-- 5. CATEGORY TABLE

```
create table category (  
    category_id int identity(1,1) primary key,  
    event_id int not null,  
    category_name varchar(100) not null,  
    description varchar(300) null,  
  
    constraint fk_category_event  
        foreign key (event_id)  
        references event(event_id),  
  
    constraint uq_event_category  
        unique (event_id, category_name)  
);
```

-- 6. REGISTRATION / ENROLMENT TABLE

```
create table registration (  
    registration_id int identity(1,1) primary key,  
    event_id int not null,  
    participant_id int not null,
```

```
registration_date datetime not null default getdate(),  
status varchar(20) not null default 'registered',  
payment_status varchar(20) not null default 'pending',  
amount_paid decimal(10,2) not null default 0.00,
```

```
constraint fk_registration_event
```

```
foreign key (event_id)  
references event(event_id),
```

```
constraint fk_registration_participant
```

```
foreign key (participant_id)  
references participant(participant_id),
```

```
constraint uq_event_participant
```

```
unique (event_id, participant_id),
```

```
constraint chk_registration_status
```

```
check (status in ('registered', 'cancelled', 'completed')),
```

```
constraint chk_payment_status
```

```
check (payment_status in ('pending', 'paid', 'refunded')),
```

```
constraint chk_amount_paid
```

```
check (amount_paid >= 0)
```

```
);
```

-- 7. RESULT TABLE

```
create table result (  
    result_id int identity(1,1) primary key,  
    event_id int not null,  
    participant_id int not null,  
    category_id int not null,  
    position int not null,  
    finish_time time not null,  
    status varchar(20) not null default 'finished',  
    points int not null default 0,  
  
    constraint fk_result_event  
        foreign key (event_id)  
        references event(event_id),  
  
    constraint fk_result_participant  
        foreign key (participant_id)  
        references participant(participant_id),  
  
    constraint fk_result_category  
        foreign key (category_id)  
        references category(category_id),  
  
    constraint chk_result_position
```

```
        check (position > 0),

constraint chk_result_points

        check (points >= 0),

constraint chk_result_status

        check (status in ('finished', 'dnf', 'dns'))
);

-- INSERT SAMPLE USERS

insert into [user]

    (username, password_hash, role, email)

values

    ('raceadmin1', 'hashed_password_001', 'organiser',

    'thabo@raceday.co.za'),

    ('raceadmin2', 'hashed_password_002', 'organiser',

    'nomsa@raceday.co.za'),

    ('runner01', 'hashed_password_003', 'participant',

    'sipho@gmail.com'),

    ('runner02', 'hashed_password_004', 'participant',
```

```
'lerato@gmail.com');
```

```
-- INSERT SAMPLE ORGANISERS
```

```
insert into organiser
```

```
(user_id, organiser_name, email, phone, organisation)
```

```
values
```

```
(1, 'Thabo Mokoena', 'thabo@raceday.co.za',  
'0825551234', 'Johannesburg Road Runners'),
```

```
(2, 'Nomsa Dlamini', 'nomsa@raceday.co.za',  
'0835556789', 'Cape Active Events');
```

```
-- INSERT SAMPLE PARTICIPANTS
```

```
insert into participant
```

```
(user_id, full_name, email, cell_number, date_of_birth, gender)
```

```
values
```

```
(3, 'Sipho Nkosi', 'sipho@gmail.com',  
'0712345678', '1998-04-15', 'Male'),
```

```
(4, 'Lerato Maseko', 'lerato@gmail.com',  
'0723456789', '2000-09-22', 'Female');
```

```
-- INSERT THREE EVENTS
```

insert into event

(organiser_id, event_name, event_type, description,
event_date, location, start_time, end_time, status)

values

(1,
'Soweto Community Run',
'Running',
'A community road running event through Soweto.',
'2026-10-10',
'Soweto, Johannesburg',
'07:00',
'12:00',
'upcoming'),

(2,
'Cape Town Cycle Challenge',
'Cycling',
'A scenic cycling event around Cape Town.',
'2026-11-15',
'Cape Town',
'06:00',
'14:00',
'upcoming'),

(1,
'Durban Beach Walk',

```
'Walking',  
'A family-friendly charity walk along the Durban beachfront.',  
'2026-12-05',  
'Durban',  
'08:00',  
'11:00',  
'upcoming');
```

```
-- INSERT CATEGORIES FOR EVENT 1
```

```
insert into category
```

```
(event_id, category_name, description)
```

```
values
```

```
(1, '10 km Open', '10 kilometre open running category'),
```

```
(1, '10 km Junior', '10 kilometre junior running category');
```

```
-- INSERT CATEGORIES FOR EVENT 2
```

```
insert into category
```

```
(event_id, category_name, description)
```

```
values
```

```
(2, '50 km Open', '50 kilometre open cycling category'),
```

```
(2, '100 km Open', '100 kilometre open cycling category');
```

```
-- INSERT CATEGORIES FOR EVENT 3
```


insert into category

(event_id, category_name, description)

values

(3, '5 km Family', '5 kilometre family walking category'),

(3, '10 km Charity', '10 kilometre charity walking category');

-- INSERT SAMPLE ENROLMENTS

insert into registration

(event_id, participant_id, registration_date,

status, payment_status, amount_paid)

values

(1, 1, '2026-09-01 09:30:00',

'registered', 'paid', 150.00),

(1, 2, '2026-09-02 10:15:00',

'registered', 'paid', 150.00),

(2, 1, '2026-09-03 11:00:00',

'registered', 'paid', 350.00),

(2, 2, '2026-09-03 14:20:00',

'registered', 'pending', 0.00),

(3, 1, '2026-09-04 08:45:00',

```
'registered', 'paid', 100.00),
```

```
(3, 2, '2026-09-04 09:10:00',
```

```
'registered', 'paid', 100.00);
```

```
-- INSERT SAMPLE RESULTS
```

```
insert into result
```

```
(event_id, participant_id, category_id,
```

```
position, finish_time, status, points)
```

```
values
```

```
(1, 1, 1, 1, '00:42:15', 'finished', 100),
```

```
(1, 2, 1, 2, '00:45:32', 'finished', 90);
```

```
-- VERIFY THE DATABASE
```

```
select * from [user];
```

```
select * from organiser;
```

```
select * from participant;
```

```
select * from event;
```

```
select * from category;
```

```
select * from registration;
```

```
select * from result;
```

User Table:

	user_id	username	password_hash	role	email	created_at
1	1	raceadmin1	hashed_password_001	organiser	thabo@raceday.co.za	2026-09-04 18:47:36.913
2	2	raceadmin2	hashed_password_002	organiser	nomsa@raceday.co.za	2026-09-04 18:47:36.913
3	3	runner01	hashed_password_003	participant	sipho@gmail.com	2026-09-04 18:47:36.913
4	4	runner02	hashed_password_004	participant	lerato@gmail.com	2026-09-04 18:47:36.913

Organizer Table:

	organiser_id	user_id	organiser_name	email	phone	organisation
1	1	1	Thabo Mokoena	thabo@raceday.co.za	0825551234	Johannesburg Road Runners
2	2	2	Nomsa Dlamini	nomsa@raceday.co.za	0835556789	Cape Active Events

Participant Table:

	participant_id	user_id	full_name	email	cell_number	date_of_birth	gender
1	1	3	Sipho Nkosi	sipho@gmail.com	0712345678	1998-04-15	Male
2	2	4	Lerato Maseko	lerato@gmail.com	0723456789	2000-09-22	Female

Event Table:

	event_id	organiser_id	event_name	event_type	description	event_date	location	start_time	end_time	status
1	1	1	Soweto Community Run	Running	A community road running event through Soweto.	2026-10-10	Soweto, Johannesburg	07:00:00.0000000	12:00:00.0000000	upcoming
2	2	2	Cape Town Cycle Challenge	Cycling	A scenic cycling event around Cape Town.	2026-11-15	Cape Town	06:00:00.0000000	14:00:00.0000000	upcoming
3	3	1	Durban Beach Walk	Walking	A family-friendly charity walk along the Durban b...	2026-12-05	Durban	08:00:00.0000000	11:00:00.0000000	upcoming

Category Table:

	category_id	event_id	category_name	description
1	1	1	10 km Open	10 kilometre open running category
2	2	1	10 km Junior	10 kilometre junior running category
3	3	2	50 km Open	50 kilometre open cycling category
4	4	2	100 km Open	100 kilometre open cycling category
5	5	3	5 km Family	5 kilometre family walking category
6	6	3	10 km Charity	10 kilometre charity walking category

Registration Table:

	registration_id	event_id	participant_id	registration_date	status	payment_status	amount_paid
1	1	1	1	2026-09-01 09:30:00.000	registered	paid	150.00
2	2	1	2	2026-09-02 10:15:00.000	registered	paid	150.00
3	3	2	1	2026-09-03 11:00:00.000	registered	paid	350.00
4	4	2	2	2026-09-03 14:20:00.000	registered	pending	0.00
5	5	3	1	2026-09-04 08:45:00.000	registered	paid	100.00
6	6	3	2	2026-09-04 09:10:00.000	registered	paid	100.00

Result Table:

	result_id	event_id	participant_id	category_id	position	finish_time	status	points
1	1	1	1	1	1	00:42:15.0000000	finished	100
2	2	1	2	1	2	00:45:32.0000000	finished	90

Reference

Coronel, C. and Morris, S. (2019) *Database systems: design, implementation, & management*. 13th edn. Boston: Cengage Learning.